

AI ASSI CODING

HTNO:2303A510B1

Assignment – 20.1

Batch: 14

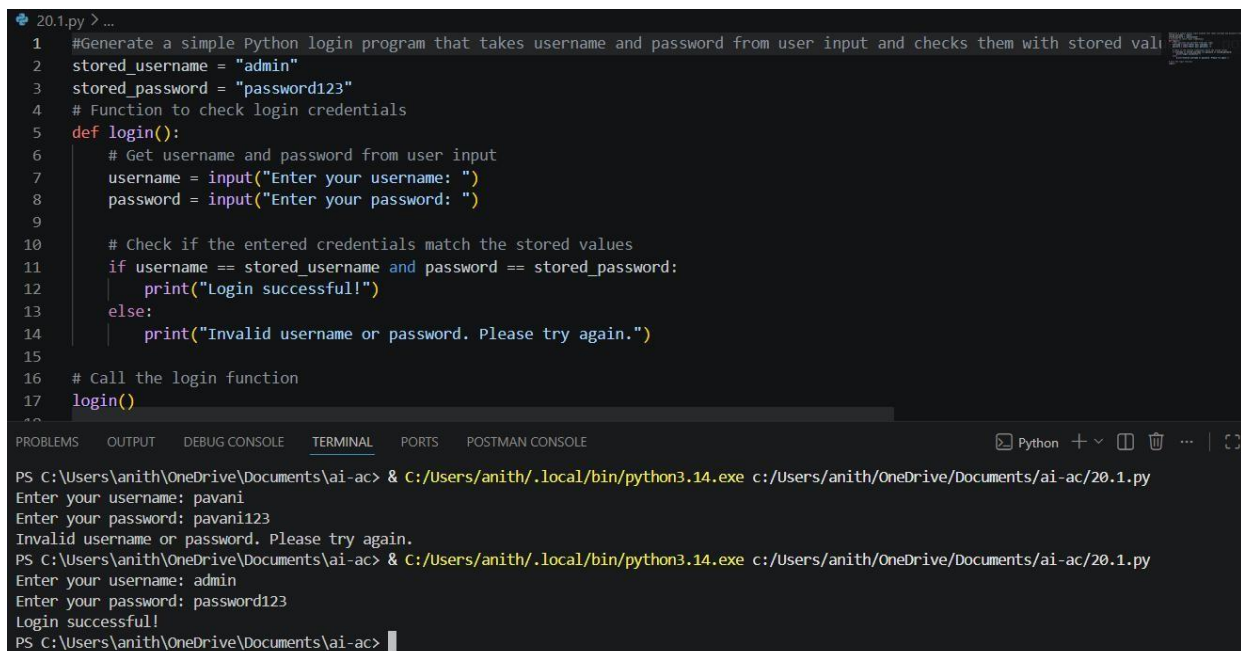
Lab 20 – Security Testing: Identifying Vulnerabilities in AI-Generated Code

Task 1 – Input Validation Check

PROMPT:

Generate a simple Python login program that takes username and password from user input and checks them with stored values. Do not include input validation.

CODE AND OUTPUT:



```
20.1.py > ...
1 #Generate a simple Python login program that takes username and password from user input and checks them with stored val
2 stored_username = "admin"
3 stored_password = "password123"
4 # Function to check login credentials
5 def login():
6     # Get username and password from user input
7     username = input("Enter your username: ")
8     password = input("Enter your password: ")
9
10    # Check if the entered credentials match the stored values
11    if username == stored_username and password == stored_password:
12        print("Login successful!")
13    else:
14        print("Invalid username or password. Please try again.")
15
16    # Call the login function
17    login()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
PS C:\Users\anith\OneDrive\Documents\ai-ac> & C:/Users/anith/.local/bin/python3.14.exe c:/Users/anith/OneDrive/Documents/ai-ac/20.1.py
Enter your username: pavani
Enter your password: pavani123
Invalid username or password. Please try again.
PS C:\Users\anith\OneDrive\Documents\ai-ac> & C:/Users/anith/.local/bin/python3.14.exe c:/Users/anith/OneDrive/Documents/ai-ac/20.1.py
Enter your username: admin
Enter your password: password123
Login successful!
PS C:\Users\anith\OneDrive\Documents\ai-ac> |
```

EXPLANATION:

1. The login program takes username and password without checking their format.
2. This allows invalid or malicious inputs (like special characters or scripts).
3. Lack of validation can lead to security issues like injection attacks.
4. Input validation ensures data follows specific rules (length, characters, etc.).
5. Using re (regular expressions) improves security by restricting allowed input patterns.

Task 2 – SQL Injection Prevention

PROMPT:

Generate a Python script using SQLite that fetches user details from a database using username input. Use simple string concatenation in the SQL query.

CODE AND OUTPUT:

```
18 '''  
19 #Generate a Python script using SQLite that fetches user details from a database using username input. Use simple string concatenation in the SQL query.  
20 import sqlite3  
21 # Connect to the SQLite database (or create it if it doesn't exist)  
22 conn = sqlite3.connect('users.db')  
23 # Create a cursor object to interact with the database  
24 cursor = conn.cursor()  
25 # Create a table to store user details (if it doesn't already exist)  
26 cursor.execute('''CREATE TABLE IF NOT EXISTS users (  
27 id INTEGER PRIMARY KEY AUTOINCREMENT,  
28   
29 username TEXT NOT NULL,  
30 password TEXT NOT NULL,  
31 email TEXT NOT NULL)''')  
32 # Insert a sample user into the database (for testing purposes)  
33 cursor.execute("INSERT INTO users (username, password, email) VALUES ('admin', 'password123', 'admin@example.com')")  
34 # Commit the changes to the database  
35 conn.commit()  
36 # Function to fetch user details based on username input  
37 def fetch_user_details():  
38 # Get username from user input  
39 username = input("Enter the username to fetch details: ")  
40 # Create a SQL query using string concatenation (not recommended for production code)  
41 query = "SELECT * FROM users WHERE username = " + username + ""  
42 # Execute the query  
43 cursor.execute(query)  
44 # Fetch the user details  
45 user_details = cursor.fetchone()  
46 # Check if user details were found and print them  
47 if user_details:  
48 print("User Details:")  
49 print("ID:", user_details[0])  
50 print("Username:", user_details[1])  
51 print("Password:", user_details[2])  
52 print("Email:", user_details[3])  
53 else:  
54 print("No user found with the username:", username)  
55 # Call the function to fetch user details  
56 fetch_user_details()  
57 # Close the database connection  
58 conn.close()  
59 '''  
60
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS POSTMAN CONSOLE

```
PS C:\Users\anith\OneDrive\Documents\ai-ac> & C:/Users/anith/.local/bin/python3.14.exe c:/Users/anith/OneDrive/Documents/ai-ac/20.1.py  
Enter the username to fetch details: admin  
User Details:  
ID: 1  
Username: admin  
Password: password123  
Email: admin@example.com
```

EXPLANATION:

1. The program builds SQL queries using string concatenation.
2. This allows attackers to manipulate queries using malicious input.
3. Example: ' OR '1'='1 can bypass authentication.
4. This vulnerability is called SQL Injection.
5. Using parameterized queries (?) prevents injection by separating data from query.

Task 3 – Cross-Site Scripting (XSS) Check

PROMPT:

Convert this SQL query into an equivalent Pandas DataFrame operation in Python.

CODE:

```

1 #Generate a simple HTML feedback form with JavaScript that takes user input and displays it on the page.Do not include any input sanitization.
2 <!DOCTYPE html>
3 <html lang="en">
4 <head>
5   <meta charset="UTF-8">
6   <meta name="viewport" content="width=device-width, initial-scale=1.0">
7   <title>Feedback Form</title>
8 </head>
9 <body>
10  <h1>Feedback Form</h1>
11  <form id="feedbackForm">
12    <label for="name">Name:</label><br>
13    <input type="text" id="name" name="name"><br><br>
14    <label for="feedback">Feedback:</label><br>
15    <textarea id="feedback" name="feedback"></textarea><br><br>
16    <button type="button" onclick="submitFeedback()">Submit</button>
17  </form>
18
19  <h2>Your Feedback:</h2>
20  <div id="displayFeedback"></div>
21
22  <script>
23    function submitFeedback() {
24      var name = document.getElementById('name').value;
25      var feedback = document.getElementById('feedback').value;
26      var displayDiv = document.getElementById('displayFeedback');
27      displayDiv.innerHTML = "<p><strong>Name:</strong> " + name + "</p><p><strong>Feedback:</strong> " + feedback + "</p>";
28    }
29  </script>
30 </body>
31 </html>

```

OUTPUT:

Feedback Form

Name:

Feedback:

Submit

Your Feedback:

EXPLANATION:

1. The web form displays user input directly using innerHTML.
2. This allows execution of malicious scripts entered by users.
3. Example: `<script>alert('Hacked')</script>` runs in browser.
4. This vulnerability is called XSS (Cross-Site Scripting).
5. Using `textContent` or escaping input prevents script execution.

Task4:Real-Time App: Security Audit of AI-Generated Code

PROMPT:

Generate a simple web application login system (frontend + backend) with basic authentication.

CODE:

```

#Generate a simple web application login system (frontend + backend) with basic authentication.
/*Frontend (HTML + JavaScript):
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Login System</title>
</head>
<body>
  <h1>Login</h1>
  <form id="loginForm">
    <label for="username">Username:</label><br>
    <input type="text" id="username" name="username"><br><br>
    <label for="password">Password:</label><br>
    <input type="password" id="password" name="password"><br><br>
    <button type="button" onclick="login()">Login</button>
  </form>

  <div id="loginResult"></div>

  <script>
    function login() {
      var username = document.getElementById('username').value;
      var password = document.getElementById('password').value;

      fetch('/login', {
        method: 'POST',
        headers: {
          'Content-Type': 'application/json'
        },
        body: JSON.stringify({ username: username, password: password })
      })
      .then(response => response.json())
      .then(data => {
        var resultDiv = document.getElementById('loginResult');
        if (data.success) {
          resultDiv.innerHTML = "<p>Login successful! Welcome, " + data.username + "</p>";
        } else {
          resultDiv.innerHTML = "<p>Login failed. Please try again.</p>";
        }
      });
    }
  </script>
</body>
</html>

```

OUTPUT:

Login

Username:

Password:

Task 5: Insecure Data Serialization Check

PROMPT:

Generate a Python script that saves and loads user data using pickle.

CODE and OUTPUT:

```

56 #Generate a Python script that saves and loads user data using pickle.
57 import pickle
58 # Function to save user data to a file using pickle
59 def save_user_data(user_data, filename):
60     with open(filename, 'wb') as file:
61         pickle.dump(user_data, file)
62         print("User data saved successfully.")
63 # Function to load user data from a file using pickle
64 def load_user_data(filename):
65     try:
66         with open(filename, 'rb') as file:
67             user_data = pickle.load(file)
68             print("User data loaded successfully.")
69             return user_data
70     except FileNotFoundError:
71         print("File not found. Returning an empty dictionary.")
72         return {}
73 # Example usage
74 if __name__ == "__main__":
75     # Sample user data to save
76     user_data = {
77         'username': 'admin',
78         'password': 'password123',
79         'email': 'admin@example.com'
80     }
81     # Save the user data
82     save_user_data(user_data, 'user_data.pkl')
83     # Load the user data
84     loaded_data = load_user_data('user_data.pkl')
85     print("Loaded User Data:", loaded_data)
86
87

```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS POSTMAN CONSOLE

```

PS C:\Users\anith\OneDrive\Documents\ai-ac> & 'c:\Users\anith\local\bin\python3.14.exe' 'c:\Us
ments\ai-ac\20.1.py'
User data saved successfully.
User data loaded successfully.
Loaded User Data: {'username': 'admin', 'password': 'password123', 'email': 'admin@example.com'}

```

EXPLANATION:

1. The program uses pickle to store and load data.
2. pickle.load() can execute malicious code if input is untrusted.
3. This makes it unsafe for handling external data.
4. Attackers can exploit it to run harmful commands.
5. Using JSON is safer as it only stores data, not executable code.